Agenda

- Exercise - Comparing different classifiers
- Recap: The holdout method for model selection / parameter tuning
- Preprocessing of numerical and categorical data
  - Dealing with missing values and outliers
  - Encoding categorical data
  - Scaling
- Feature selection
  - Filtering methods
  - Wrappers
  - Embedded methods

Model selection / Parameter tuning

Holdout method with Test/Validation/Train split

Holdout method

- The **holdout method** is the simplest idea for model selection
  - Split into a **training set (for model training)** and a **test set (for model evaluation)**
  - **Problem:** The test set becomes part of the model selection if we repeat this over many models! This is an instance of **information leakage/bleeding.** It can lead to overestimation of the model performance.
- A better version of the holdout method (*always do this!*):
  - Split into a **training set (for model training),** a **validation set (for model evaluation)** and a **test set (for the final performance evaluation)**
  - After we have selected the best model, we typically retrain on the training set combined with the validation set

Holdout method – issue

- Sensitive to how we do the training/validation/test set split
  - One remedy (that we will use for CA3) is to average over several different random splits
  - `03_knn_hyperparameters_validation_set.ipynb`
  - Also see `CA3_Workflow.pdf`

- Later on, you will learn a more robust method for model selection: *k-fold cross-validation*

Preprocessing and Feature Selection

Building Good Training Sets

See Ch. 04 in the course book



Building Good Training Sets – Overview

- **Removing** or **imputing** missing values in the data set
- Handling **categorical** data ("*feature encoding*")
- Bringing **features** onto the same **scale** ("*feature scaling*")

- **Selecting meaningful features** ("*feature selection*")
- Algorithms for feature selection
- **Assessing feature importance** with random forests

Preprocessing and Feature Selection

Preprocessing data

Building Good Training Sets

See Ch. 04 in the course book



Building Good Training Sets – Overview

- **Key factors** that determine **how well** a machine learning algorithm can learn:
  - **Quality** of the data
  - **Amount** of **useful information** that the data contains

- It is **critical (!!)** that we **examine** and **preprocess** a dataset **before** we feed it to a machine learning algorithm
- In the following, we will discuss the **essential preprocessing techniques** for building datasets that are well-prepared for machine learning tasks

Dealing with missing values – Removal

- NaN: Not a Number
- NULL

	A	B	C	D
0	1.00	2.00	3.00	4.00
1	5.00	6.00	nan	8.00
2	10.00	11.00	12.00	nan

`04_missing_data.ipynb`

Dealing with missing values – Removal

- First solution: Remove samples or features with NaN values

	A	B	C	D
0	1.00	2.00	3.00	4.00
1	5.00	6.00	nan	8.00
2	10.00	11.00	12.00	nan



	A	B
0	1.00	2.00
1	5.00	6.00
2	10.00	11.00

```
df.dropna(axis=1)
```

- Problem for small data sets or in the case of bad data quality:
  - We might **remove too much** of the data and don't have enough information left to train a good machine learning model

`04_missing_data.ipynb`

Dealing with missing values – Imputation

- Replace the missing value with a reasonable value so we can keep samples that maybe only have a small number of missing feature values
- Scikit-learn: `SimpleImputer`: mean, most frequent value, median

	A	B	C	D
0	1.00	2.00	3.00	4.00
1	5.00	6.00	nan	8.00
2	10.00	11.00	12.00	nan



	0	1	2	3
0	1.00	2.00	3.00	4.00
1	5.00	6.00	7.50	8.00
2	10.00	11.00	12.00	6.00

```
SimpleImputer(
    strategy='mean',
)
```

`04_missing_data.ipynb`

Dealing with outliers

- Difficult task since it is very **dataset-specific**
- **Outliers**
  - Can result from faulty instruments or errors in the data → we may want to remove them from training (and prediction). Requires domain/expert knowledge about the type of data.
    - Example: A weight measurement of an object must be positive

      → **Remove** samples/features **manually** after expert judgement
  - Can represent **rare** data points
    1. We might not be interested in having a good performance for rare unimportant events
    2. The opposite can also be the case: We want to perform well on outliers as well because they represent some rare but **important events**

Dealing with outliers



- In the case **we are sure** that **we are not interested** in **performing well on outliers**, we may use the Z-score to remove all outliers that are, for example, 3 sample standard deviations away from the mean. However, note that this method only **works well** for **normally distributed data.**
- **Z-score:** ***(x – mean)/std***
  - Where *x*: feature value; mean: mean of the feature values; std: standard deviation of the feature values (values in one column)
  - Can be positive or negative
  - A Z-score with absolute value of 3 indicates very unlikely events

`04_outliers_zscore.ipynb`

Preprocessing and Feature Selection

Preprocessing: Encoding categorical data

Building Good Training Sets

See Ch. 04 in the course book



Categorical features

- Features that can assume one value among a finite set of values
- Examples:
  - T-shirt size: {“S”, “M”, “L”, “XL”} (ordinal)
  - Patient is wearing a T-shirt?: {“True”, “False”} (binary, nominal)
  - T-shirt colour: {“Red”, “Blue”, “Green”} (nominal)
- **Ordinal features:** Sets that have an intrinsic order (e.g. T-shirt size)
- **Nominal features:** Sets that have no order (e.g. T-shirt colour)

Categorical features

Example data set with **nominal** and **ordinal** categorical features
(and a numerical feature which is also ordinal)

	color (nominal)	size (ordinal)	price (numeric & ordinal)	class label (nominal)
0	green	M	10.100000	class1
1	red	L	13.500000	class2
2	blue	XL	15.300000	class1

`04_categorical_data_encoding.ipynb`

Mapping ordinal features

Data with an **inherent order (ordinal data)** can be **converted** to **numerical integer data (A)**. If we don't want to weigh/quantify the order by a number, we can also use **a threshold encoding (B)** approach using multiple new features (e.g. "x > M" + "x > L" each {0,1})

	color	size	price	class label
0	green	M	10.100000	class1
1	red	L	13.500000	class2
2	blue	XL	15.300000	class1

	color	size	price	class label
0	green	1	10.100000	class1
1	red	2	13.500000	class2
2	blue	3	15.300000	class1

(A)

```
size_mapping = {'XL': 3, 'L': 2, 'M': 1}
df['size'] = df['size'].map(size_mapping)
```

```
# Use a threshold approach for the size feature
df2['x > M'] = df2['size'].map(lambda x: 0 if x == 'M' else 1)
df2['x > L'] = df2['size'].map(lambda x: 1 if x == 'XL' else 0)
df2.drop('size', axis=1, inplace=True)
```

	color	price	class label	x > M	x > L
0	green	10.100000	class1	0	0
1	red	13.500000	class2	1	0
2	blue	15.300000	class1	1	1

(B)

04_categorical_data_encoding.ipynb

Encoding class labels

04_categorical_data_encoding.ipynb

Class labels are usually **not** ordinal, and none of the classifiers you learn about in this course consider any ordering of class labels. **Common practice: Use integers**

	color	size	price	class label
0	green	1	10.100000	class1
1	red	2	13.500000	class2
2	blue	3	15.300000	class1



	color	size	price	class label
0	green	1	10.100000	0
1	red	2	13.500000	1
2	blue	3	15.300000	0





```
class_mapping = {label: i for i, label in enumerate(np.unique(df['class label']))}
df['class label'] = df['class label'].map(class_mapping)
```

```
# Alternative to the above using sklearn's LabelEncoder
from sklearn.preprocessing import LabelEncoder

class_le = LabelEncoder()
y = class_le.fit_transform(df['class label']) # ".fit_transform" is short for ".fit" and then ".transform"
```

One-hot encoding for nominal data

- **Idea:** Replace a categorical feature with (*k*) possible states into a bit-array of size (*k-1*), where each bit represents whether the corresponding category is assumed (***hot***: 1) or not.
- Use the value of one bit (*binary digit*) $\in \{0,1\}$ (alternatively represented as $\{\text{False}, \text{True}\};\ \{\text{yes}, \text{no}\}; \{+, -\}; \{\text{on}, \text{off}\}$).
- Only *(k-1)* features are needed because the last state is implicitly defined by all bits equal to 0.
- In statistics, the new features are also known as "**dummy variables**".

(the "hot" bit is framed in green)

	color	size	price	class label
0	green	1	10.100000	0
1	red	2	13.500000	1
2	blue	3	15.300000	0

	size	price	class label	color_green	color_red
0	1	10.100000	0	True	False
1	2	13.500000	1	False	True
2	3	15.300000	0	False	False

blue

```
df_dum = pd.get_dummies(df, drop_first=True)
show_dataset(df_dum)
```

OR

```
color_ohe = OneHotEncoder(drop='first', sparse_output=False)
out = color_ohe.fit_transform(df[['color']]) # expects a 2D array
```

04_categorical_data_encoding.ipynb

Preprocessing and Feature Selection

Preprocessing: Scaling data

Building Good Training Sets

See Ch. 04 in the course book

Scaling data

- Two main ideas: **(1)** Scale to the **same range [min, max]** **(2) Standardize**
- Scikit-learn: (1) `MinMaxScaler`, (2) `StandardScaler`
- Min/Max-scaling is sensitive to outliers

(1) $$X_{\text{scaled}} = \frac{X - \min(X)}{\max(X) - \min(X)}$$

(2) $$X_{\text{scaled}} = \frac{(X - \mu)}{\sigma}$$

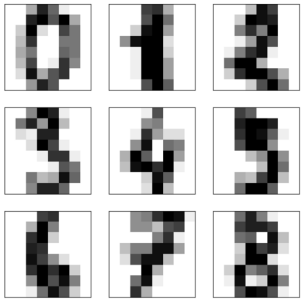
X_{scaled} are the new, transformed column values (a column-vector)
X is the original values
μ vector of the means of each column
σ vector of the standard deviations of each column

} feature-wise scaling

`04_scaling.ipynb`

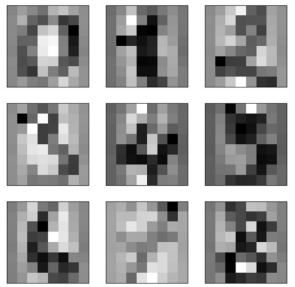
Scaling data

- Scaling each feature separately (as we did so far in the course) is usually a good idea. But depending on the data and the chosen ML model, we need to be careful.
- An **example** is **image data**: E.g. for the handwritten digits to the right, the intensity of each pixel is a feature (8*8 = 64 features per sample).
  - All pixel values use the same scale/unit and are comparable
  - Pixel values are correlated depending on the position! Independent scaling leads to loss of this information!
  - **In this case:** Use mean/std of the whole data set (all pixels in all samples) for scaling instead of feature-wise independent scaling
- A classifier that treats each pixel independently (Logistic Regression) may not be affected by feature-wise scaling, while a classifier that can make use of differences between features (e.g. Kernel SVM) could be harmed by feature-wise standardization in this case



good

Feature-wise Standardization

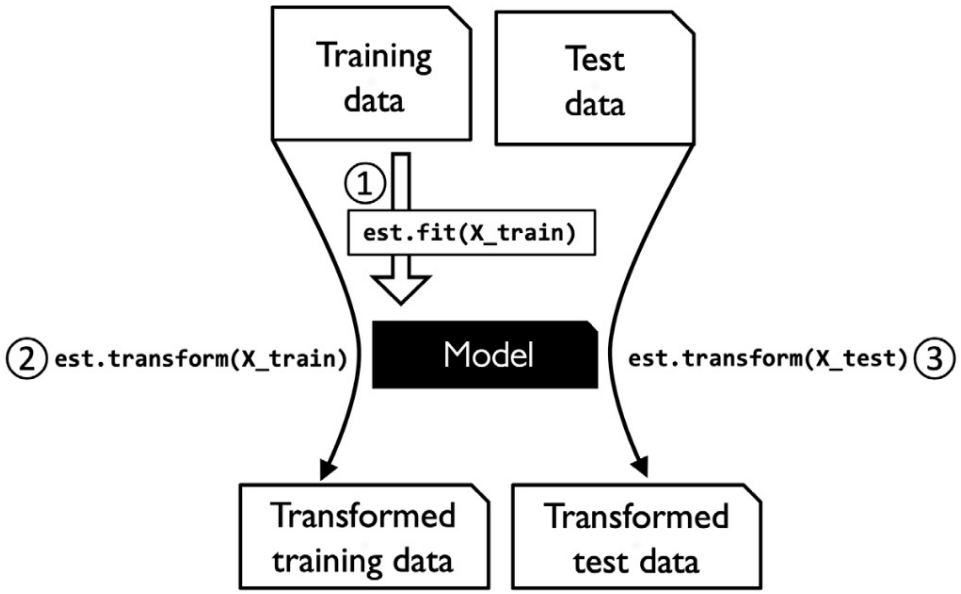


bad

`04_scaling_digits.ipynb`

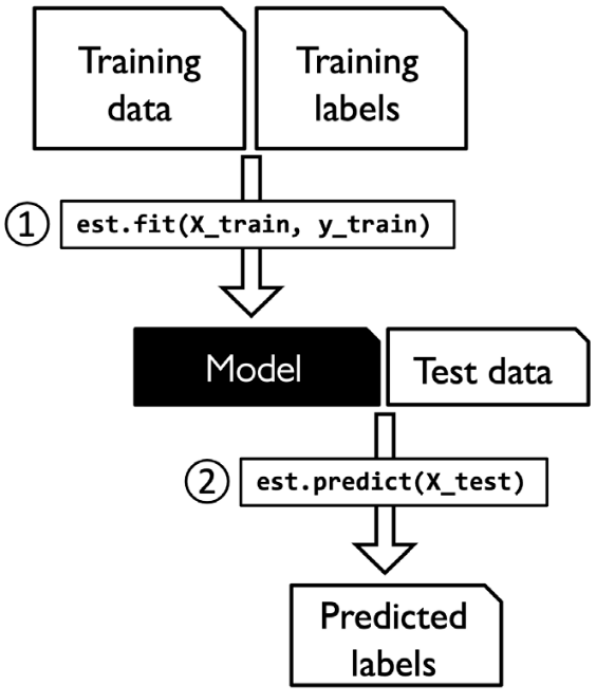
Transformer API and Estimator API (scikit-learn)

“Transformer”



Remark: The alternative interface `est.fit_transform(X_train)` performs both `fit` and `transform` of the training set (1+2) in one function call. Subsequent calls to `transform` have the same effect as if the first call would have been only `fit`.

“Estimator”



Figures from Python Machine Learning (Raschka & Mirjalili)

Preprocessing and Feature Selection

Feature Selection

Building Good Training Sets

See Ch. 04 in the course book

Feature Selection

- If a model performs **much better** on the **training set** than on the **test set**, this may be a **strong indication** of overfitting
- **Overfitting** [recap]
  - The model fits the training data too closely, focusing on particular observations in the training dataset
  - The model is too complex (too many degrees of freedom) for the given training data
  - The model does not generalise well to new data
  - The model has a high variance
- **Remedies** for reducing **generalisation** error (and preventing overfitting) [recap]
  - Collect **more training data** (*often not possible!)*
  - Introduce a **penalty** for complexity through **regularisation**
    - E.g. ℓ_1 (LASSO), ℓ_2 (Ridge), ℓ_1 + ℓ_2 (ElasticNet)
  - Choose a **simpler** model with **fewer** parameters
  - Reduce the **dimensionality** of the data

Feature Selection

- **Feature selection** is a way of **reducing the dimensionality** by **removing features**
- A way to reduce the complexity of the model and avoid overfitting
  → Reducing an initial d-dimensional feature space to a k-dimensional feature subspace where $k < d$
- Two main ways of **dimensionality reduction:**
  - **Feature selection**
    → Select a subset of the original features (this lecture)
  - **Feature extraction** with **compression**
    → Derive information from the feature set to construct a new feature subspace
    → E.g. PCA/LDA (next topic in class)

What is a good subset of features?

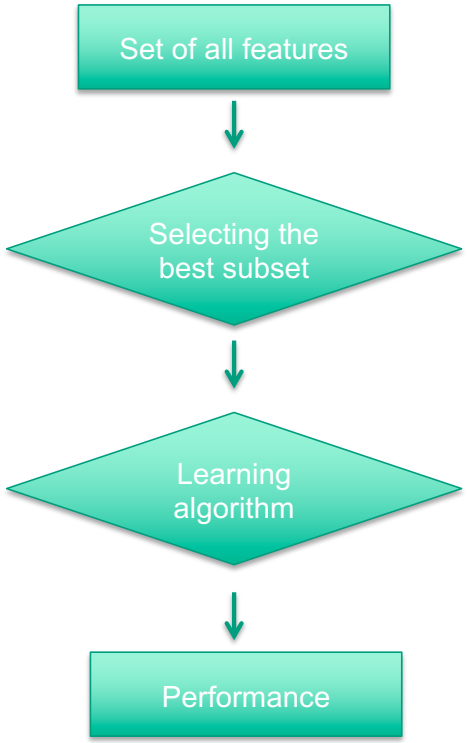
- **Relevant features:** Those that we **need** to model the data
- **Irrelevant features:** Those that are **unnecessary** to model the data
- **Redundant features:** Those that **become irrelevant** in the presence of others
- Worst mistake: To remove relevant features
- Important to **validate the predictive ability** of the learning algorithm for different feature subsets

Feature Selection methods

- Idea behind feature selection algorithms:
  - Automatically select a **subset** of features that are **most relevant** to the problem
  - Improve **computational efficiency**
  - Sometimes: **Reduce the generalisation error** of the model by removing irrelevant features or noise
- Can be divided into three main types:
  - Filtering methods
  - Wrapper methods
  - Embedded methods

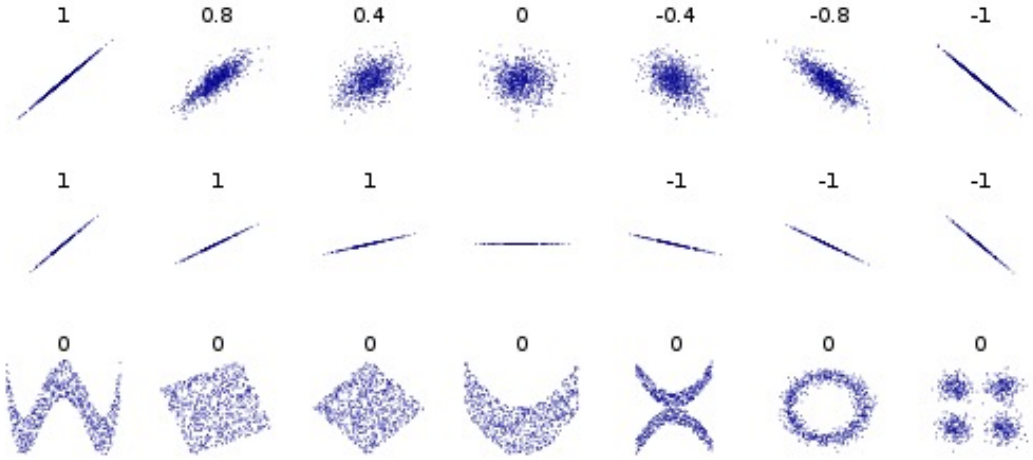
Filtering methods

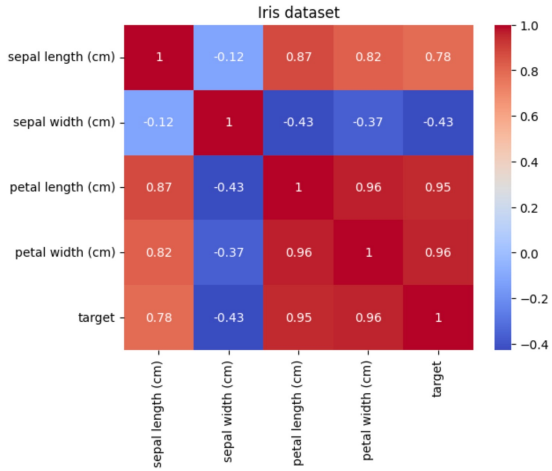
- Each feature is usually scored separately according to e.g. the correlation with the response variable
- Independent of the learning algorithm
- **Advantages:**
  - Computationally efficient
  - Robust to overfitting
- **Disadvantages:**
  - Tend to select redundant variables
  - Interactions between variables are usually not taken into account
  - Require choice of a threshold for the filter



Feature Selection based on the correlation matrix

- **Strongly correlated** features may contain redundant information -> can be left out
- **Completely unrelated / uninformative features can be left out**
- A **crude** tool to assess this is looking at the **correlation matrix**
  - **Crude** because this **only** detects **linear** relationships!



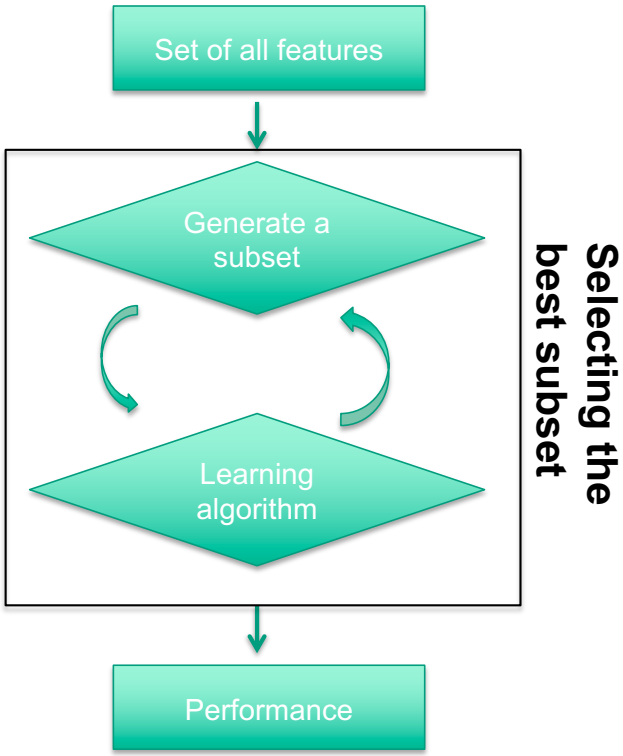


Correlation distance

`04_correlationmap.ipynb`

Wrapper methods

- Several subsets of features are searched through, and the data modelling accuracy is evaluated
- Especially useful if the algorithm doesn't support regularization
- A family of **greedy** search algorithms
- **Advantages:**
  - Allow detection of possible interactions between variables
  - Often find a good subset for a specific learning algorithm
- **Disadvantages:**
  - Risk of overfitting when the number of observations is low
  - Computationally demanding when the number of features is large



Examples of wrapper methods

- **Sequential Backward Selection:**
  - Starts with all the features
  - Iteratively removes the least significant feature
  - Stops when no improvement is observed upon removal of features
- **Sequential Forward Selection:**
  - Starts with having no feature in the model
  - Iteratively adds the feature which improves the model the most
  - Stops when no improvement is observed upon addition of features
- **Stepwise Selection:**
  - Re-considers all dropped/added variables for re-introduction/drop-out in each step

Sequential Backward Selection (SBS)



- **Sequential Backward Selection (SBS)** aims to **reduce the dimensionality** of the initial feature space with a **minimum decay** in **performance** of the classifier, to improve computational efficiency and reduce model complexity
- In certain cases, SBS **can** even **improve** the **predictive performance** of the model (if a model suffers from overfitting)
  - **Example:** k-NN typically suffers from overfitting in high dimensions due to the **curse of dimensionality**. Feature selection can help improve the **predictive performance** by reducing the feature space dimensions and therefore the susceptibility to overfitting

Sequential Backward Selection (SBS)



Main idea behind the SBS algorithm:

- SBS **sequentially removes features** from the full feature set d until no improvement is seen, or the new feature subspace contains a desired number of k features ($k < d$)
- **Which feature** is to be removed at each stage? → need to **define a loss function** *L* that is to be minimized
- The loss function can simply be the **decrease in performance** of a classifier caused by the **removal of a particular feature**
- The **feature to be removed** at each stage would be the feature that minimizes this loss → in each stage we **eliminate the feature** that **causes the least performance loss** after removal (negative performance loss would mean we increase performance!)

Sequential Forward Selection (SFS)



The SFS algorithm (versus SBS)

- Same concept as SBS, but instead of removing features we start with no features and sequentially add features. We add the feature that improves the model performance the most.
- The default “flavour” of the algorithm for Scikit-learn’s `SequentialFeatureSelector` is `direction=forward`

Code example:

`04_sequential_feature_selection.ipynb`

Sequential Forward Selection (SFS)

Code example conclusions:

`04_sequential_feature_selection.ipynb`

- Using **only a very small fraction** of the original features in the dataset, we obtained the same prediction accuracy on the test set
- This may indicate that those **three first selected** features **do not** provide less discriminatory information than the **original** dataset
- The performance of the k-NN increased a bit by reducing the number of features, and the size of the dataset was reduced substantially
- However, the way the dataset is split into training and test subsets may influence the results

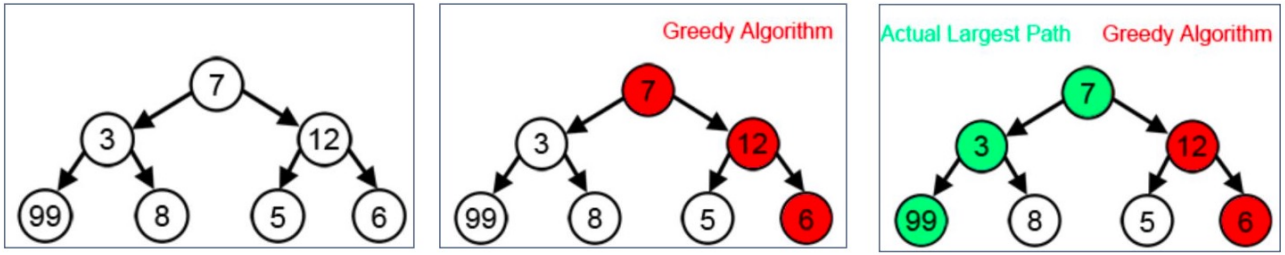
- SFS can be useful in real world applications that may involve expensive data collection steps
- Fewer features means obtaining simpler models, which are easier to interpret

Greedy versus exhaustive algorithms

- **Exhaustive** algorithms search for the best solution among all possible solutions (relation to global optimization). This can be a **prohibitively expensive** task!
- **Greedy** algorithms use ***heuristics*** to make **locally optimal** choices at **each iteration** of a **combinatorial** search solved as a **sequential** problem
- **Greedy** algorithms generally yield a **suboptimal** solution to the problem, in contrast to exhaustive search algorithms which yield the **optimal** solution (analogously to global/local minima in optimization)
- **Greedy** algorithms allow for a **less complex**, computationally **more efficient** solution

Example

"Find the largest path in a weighted directed graph"

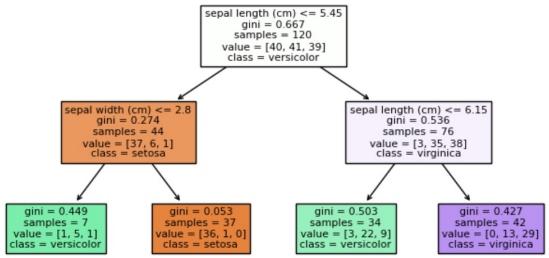


Greedy versus exhaustive algorithms

- **Exhaustive** algorithms search for the best solution among all possible solutions (relation to global optimization). This can be a **prohibitively expensive** task!
- **Greedy** algorithms use ***heuristics*** to make **locally optimal** choices at **each iteration** of a **combinatorial** search solved as a **sequential** problem
- **Greedy** algorithms generally yield a **suboptimal** solution to the problem, in contrast to exhaustive search algorithms which yield the **optimal** solution (analogously to global/local minima in optimization)
- **Greedy** algorithms allow for a **less complex**, computationally **more efficient** solution

Example 2

"Find the best splitting strategy in a decision tree"



Decision tree learning is an example of a greedy search algorithm

We find the best split locally (without evaluating whether this also results in the best split later)

Sequential feature selection in sci-kit learn

- Many more feature selection algorithms are available in Scikit-learn (not syllabus)

`sklearn.feature_selection`: Feature Selection

The `sklearn.feature_selection` module implements feature selection algorithms. It currently includes univariate filter selection methods and the recursive feature elimination algorithm.

User guide: See the Feature selection section for further details.

`feature_selection.GenericUnivariateSelect`([...])	Univariate feature selector with configurable strategy.
`feature_selection.SelectPercentile`([...])	Select features according to a percentile of the highest scores.
`feature_selection.SelectKBest`([score_func, k])	Select features according to the k highest scores.
`feature_selection.SelectFpr`([score_func, alpha])	Filter: Select the pvalues below alpha based on a FPR test.
`feature_selection.SelectFdr`([score_func, alpha])	Filter: Select the p-values for an estimated false discovery rate.
`feature_selection.SelectFromModel`(estimator, *)	Meta-transformer for selecting features based on importance weights.
`feature_selection.SelectFwe`([score_func, alpha])	Filter: Select the p-values corresponding to Family-wise error rate.
`feature_selection.SequentialFeatureSelector`(...)	Transformer that performs Sequential Feature Selection.
`feature_selection.RFE`(estimator, *[, ...])	Feature ranking with recursive feature elimination.
`feature_selection.RFECV`(estimator, *[, ...])	Recursive feature elimination with cross-validation to select features.
`feature_selection.VarianceThreshold`([threshold])	Feature selector that removes all low-variance features.

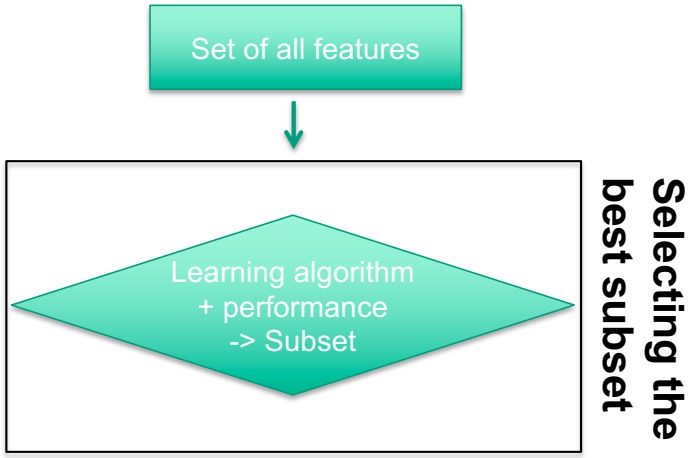
`feature_selection.chi2`(X, y)	Compute chi-squared stats between each non-negative feature and class.
`feature_selection.f_classif`(X, y)	Compute the ANOVA F-value for the provided sample.
`feature_selection.f_regression`(X, y, *[, ...])	Univariate linear regression tests returning F-statistic and p-values.
`feature_selection.r_regression`(X, y, *[, ...])	Compute Pearson's r for each features and the target.
`feature_selection.mutual_info_classif`(X, y, *)	Estimate mutual information for a discrete target variable.
`feature_selection.mutual_info_regression`(X, y, *)	Estimate mutual information for a continuous target variable.

https://scikit-learn.org/stable/api/sklearn.feature_selection.html#module-sklearn.feature_selection

https://scikit-learn.org/stable/modules/feature_selection.html#sequential-feature-selection

Embedded methods

- Feature selection is built into the learning algorithm
- **Advantages:**
  - Feature selection and learning are done simultaneously
  - Less computationally demanding than wrappers
- **Disadvantages:**
  - Less universal than wrappers
  - Feature selection performance can not be separated from learning performance



Feature Selection by regularization (recap)

See lecture slides on Overfitting and Regularization



- Add a **regularization** term to the **loss function**

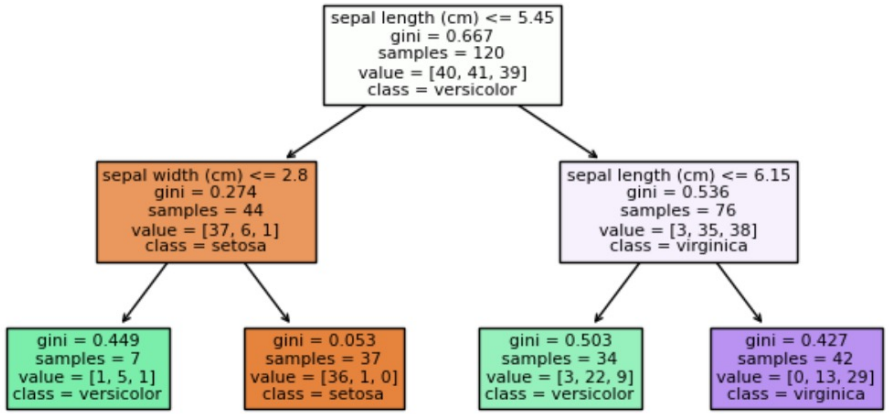
- ℓ_2 **(Ridge) – Regularization:** loss $+ \frac{\lambda}{2}||\mathbf{w}||_2^2$ $\quad ||\mathbf{w}||_2^2 = \sum_{i=1}^{m} w_i^2$
- Promotes **smaller weights** → Features corresponding to the smallest weights do not contribute significantly. For prediction, we can also omit features below a given threshold (speed-up).

- ℓ_1 **(LASSO) – Regularization:** loss $+ \lambda||\mathbf{w}||_1$ $\quad ||\mathbf{w}||_1 = \sum_{i=1}^{m} |w_i|$
- Promotes **sparsity** (**some weights become zero**) → Features corresponding to zero weights do not contribute. For prediction, these features are automatically omitted (speed-up).

Detailed discussion of ℓ_2 vs ℓ_1 : *Section 3.4, The Elements of Statistical Learning, Trevor Hastie, Robert Tibshirani, and Jerome Friedman, Springer Science+Business Media, 2009*

Feature Selection using random forests

- Train a random forest classifier on the dataset
- Measure the **feature importance** as the **averaged impurity decrease (information gain)** computed from **all decision trees** in the forest (*Impurity-based feature importance*)
  - For a given feature, over all trees
    - Check for splits that involve our feature
    - Compute the impurity decrease
    - Weight by the number of samples at the split
  - Average over all trees
  - Repeat for all features
  - Normalize so that the feature importances add up to 1



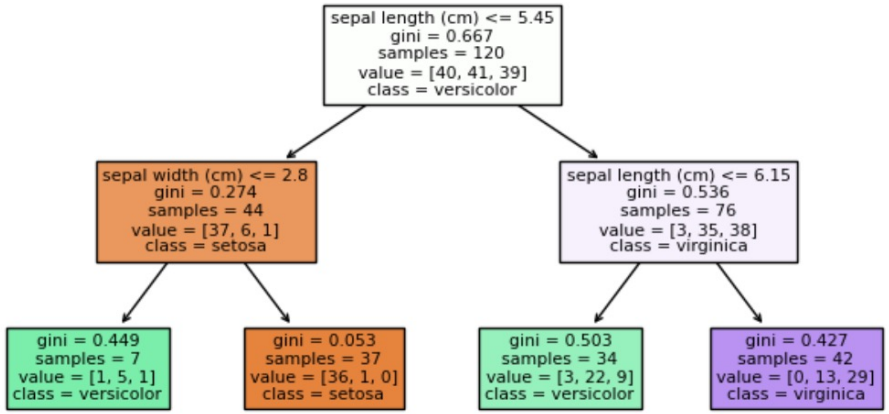
Feature Selection using random forests

The random forest classifier in scikit-learn collects the feature importance values automatically:

→ Access the `feature_importances_` attribute after fitting a `RandomForestClassifier`

Code example from Scikit-learn:

https://scikit-learn.org/stable/auto_examples/ensemble/plot_forest_importances.html



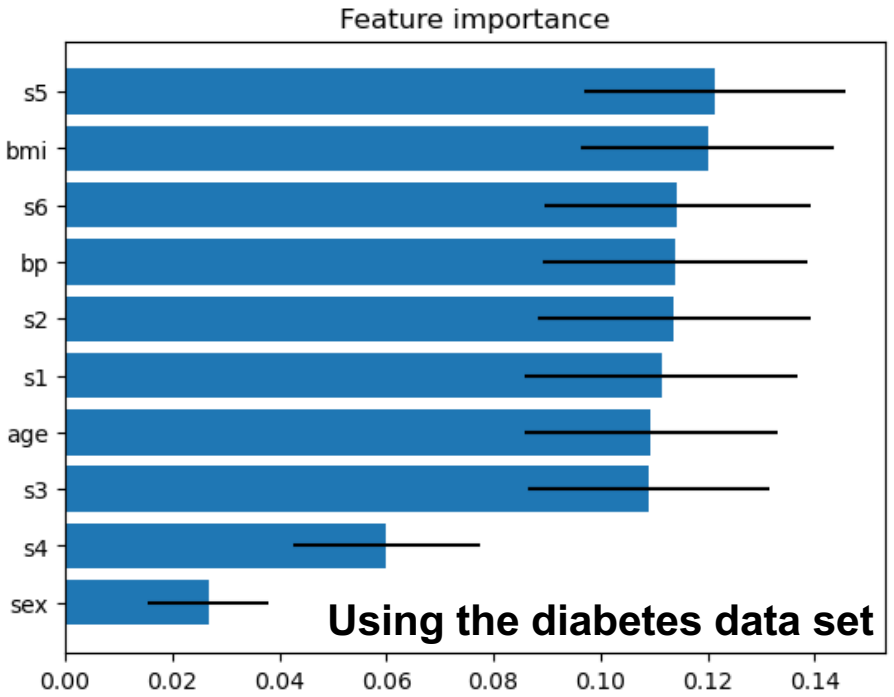
Feature Selection using random forests

The random forest classifier in scikit-learn collects the feature importance values automatically:

→ Access the `feature_importances_` attribute after fitting a `RandomForestClassifier`

Code example:

`04_randomforest_feature_importance.ipynb`



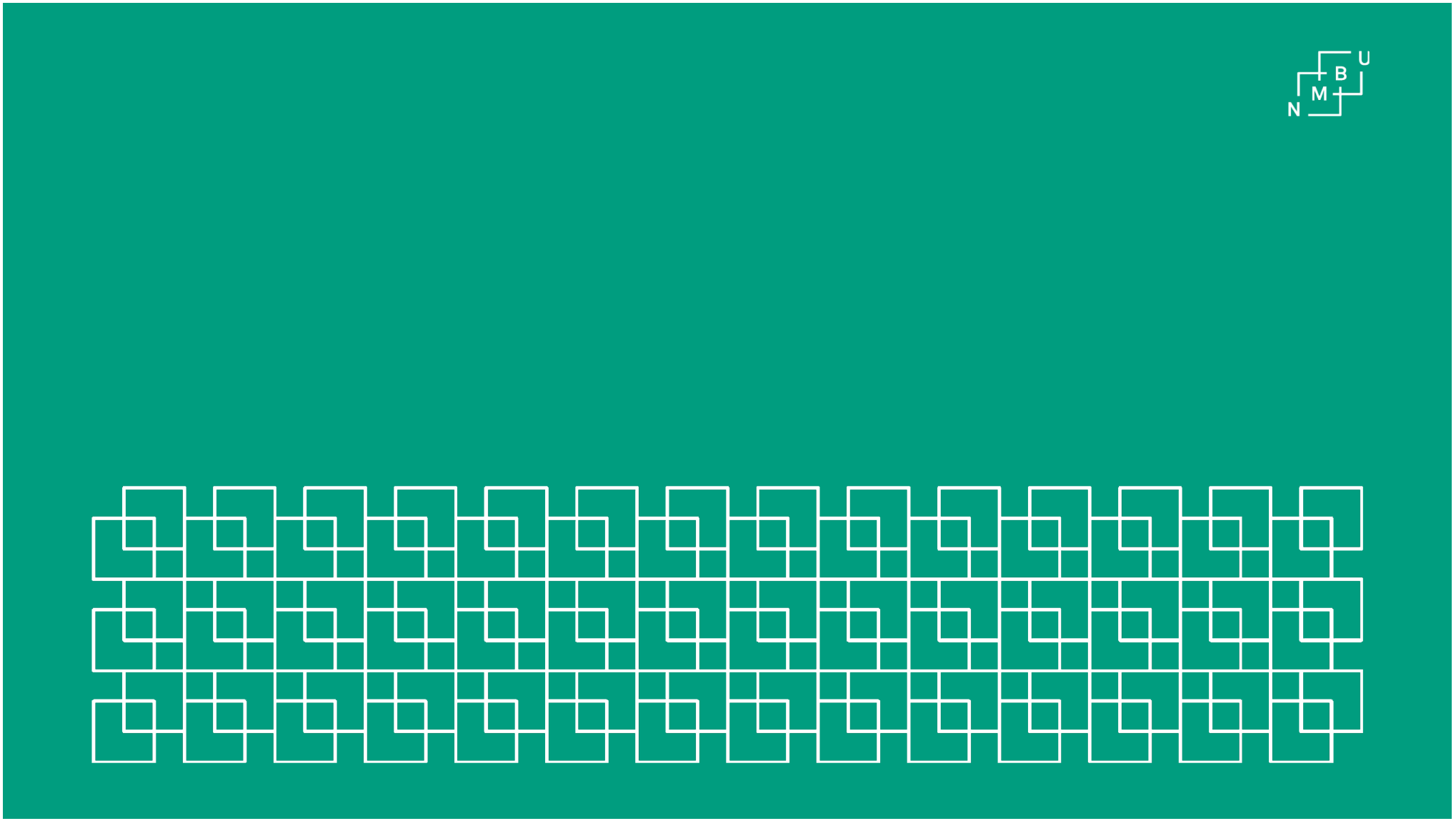
Feature Selection using random forests



- Decision trees **implicitly** assess **feature importance** by finding the best split at a given node
- Remark 1: If **two** or **more** features are **highly correlated,** the feature importance is split among the features making each feature possibly appear unimportant. This problem also occurs for categorical data with many unique values (e.g. encoded by one-hot encoding).
- Remark 2: The problem in remark 1 is important when **interpreting feature importance**. When the focus of the analysis is **on predictive performance**, it is **not so important.**
- Remark 3: There is a **more robust method** to assess feature importance in random forests called ***permutation importance*** (not syllabus). It is based on permuting the order of the values in one feature column and inspecting the out-of-bag error (mean classification error where for each sample, we only use those trees in the forest that weren't trained on that sample).

Feature Selection methods - summary

- **Filtering methods**
  - Fast and simple, but require a choice of threshold for the selection criterion
  - Do not take model performance into account
- **Wrapper methods**
  - Time consuming, since they evaluate a large number of models
  - Universal and simple, can be used with any learning method
- **Embedded methods**
  - Feature selection integrated into the modelling
  - Generally less computationally demanding than wrapper methods, but careful validation (e.g. using many different train-test-splits or cross-validation) may slow them down


N
M
B
U